

Introduction to Linux (2) — Exercises

`echo, curl, >, >>, |, scp, micro`

Nikolaj Lindberg
`nikolaj.lindberg@ling.su.se`

Part 2 of 2

Exercise 1 Log in to remote server

1. What is the command for logging into another computer (remote server)?
2. How do you log into `triton.dsv.su.se` from the terminal?
3. Log into `triton.dsv.su.se`

Exercise 2 Create/remove directory (repetition)

1. Create a directory called `dirA`
2. Create a sub-directory *in* `dirA` called `dirB`
3. Verify that the directories are there
4. Remove `dirA` and its sub-directory

Exercise 3 Create/rename directory (repetition)

1. Create a directory called `dirX`
2. Rename the directory to `dirY`
3. Remove `dirY`

Exercise 4 Copy directory (repetition)

1. Copy the directory `scripts/` from user `~kali2818` to your own home directory (on `triton.dsv.su.se`)
2. Check that it contains some files

Exercise 5 Redirect output to file

1. Use the `echo` command to create a text file, containing a single line of text (you have to come up with the text and file name yourself)
2. Use the `echo` command to *append* a line of text at the end of the same file, so that the file now has two lines of text
3. Verify the result

Exercise 6 Redirect output to another command (1/2)

Try the commands in slide

“Redirect to another command: | (pipeline)”,
in lectures notes linux_2.pdf:

1. `echo How many words? | wc -w`
2. `echo I like to shout | tr a-z A-Z`
3. `echo live | tr a-z A-Z | rev`
4. `egrep --help | less`

(No solutions to these, just try them)

Exercise 7 Redirect output to another command (2/2)

1. Count the number of files in the `/bin/` directory, by directing the output from `ls /bin/` to the command used for counting lines (hint: that command can also be used for **w**ord **c**ount)
2. What does the following command do?
`echo We are falling | tr ' ' '\n'` ← Try it out

The first argument to `tr` is a space, surrounded by single quotes.
The second argument is a code for newline, `\n`, also surrounded by quotation marks. Option-shift+7 for `\` (see `linux_commands.pdf`)

Exercise 8 Read file from a URL (internet)

NB: copy-and-paste from a PDF file may mess up the `~` character

1. Go to `https://people.dsv.su.se/~kali2818/ksl.txt` using a web-browser, just to verify that the page exists
2. Return to the terminal. Use the `curl -L` command to read (download) the file
`https://people.dsv.su.se/~kali2818/ksl.txt`
3. Run the same command, but this time write the output to a file, `ksl_poem.txt`
4. Verify that the file contains the text you saw in the browser

Exercise 9 Download a book

1. Go to `https://www.gutenberg.org/` using a web-browser
2. Find a book that you want to download (e.g.
`https://www.gutenberg.org/ebooks/5200`)
3. Click the link to the **Plain Text UTF-8** version
4. Copy the link to the plain text version (e.g.
`https://www.gutenberg.org/cache/epub/5200/pg5200.txt`)
5. Use `curl` to download the book and save in a file

Exercise 10 Copy files *between* machines (1)

1. What command is used to remotely copy files, from one computer to another?
2. Make sure there is a file in your home directory on `triton` to copy, e.g. `texts/fk.txt`
3. Log out from `triton.dsv.su.se` (see `linux_commands.pdf` for how to exit), or start a fresh terminal on the computer in front of you. Make sure you are not on `triton`.
4. Copy one of your files on `triton.dsv.su.se` to the computer in front of you

Exercise 11 Copy files *between* machines (2)

Make sure that you are not on `triton.dsv.su.se`

Create a text file on the local computer in front of you, and copy it to an existing directory in your home directory on `triton.dsv.su.se`

(This is the opposite direction from the last exercise)

Return to `triton.dsv.su.se` and verify that the copied file is there.

Exercise 12 Learn to know `micro`

`https://micro-editor.github.io/`

- ▶ Editing basics?
- ▶ Working with multiple files?
- ▶ Running a terminal (command line) in `micro`?

(No suggested solutions, you'll have to figure it out, see today's slides)

Exercise 13: Copy files and inspect script

(You might already have copied these files. In that case, skip these steps)

Start by copying some files to your home directory:

1. Copy the `/home/dsv/kali2818/texts/` directory.
(See `cp` in `linux_commands.pdf` if you've forgotten how to copy directories. Or ask somebody)
2. Copy `/home/dsv/kali2818/scripts/wf.py`
3. Open the file `wf.py` in an editor. Try to read it. Can you make any sense of it?
(You are not supposed to, this is just a warm up)

Exercise 14 Run `python3 wf.py`

1. Try the script on a small text file:

```
python3 wf.py texts/computer_world.txt
```

2. What happens? Compare the input text file to the output
3. Run the script on a larger file, e.g. `texts/s1.txt`

Exercise 15 Redirect output

When you run `python3 wf.py` on larger input files such as `texts/s1.txt`, the output doesn't fit the terminal window.

1. Redirect the output to a file called `s1_freq.txt`
2. Inspect the file, e.g. with `less` (or open it in `micro`)

Exercise 16 Multiple input files to `python3 wf.py`

Tip: The `head` command prints the first ten lines of a file

1. Run `python3 wf.py` on all text files (*.txt) in the `texts` directory
2. Redirect the output to a file `all_freq.txt`
3. Inspect and compare the top ten or so lines in `all_freq.txt` to `sl_freq.txt`
4. How do the top ten lines of the files differ?

Exercise 17 Suggest a better name

The script `wf.py` has a cryptic name on purpose, not to give away what it does before you could try it.

Typically, you want your files to have descriptive names, but not too long.

1. Suggest a better name for `wf.py`, based on what it does

Exercise 18 Call `wf.py` with nonsense argument

1. What happens if you run the script with a nonsense argument, that is not the name of a file? For example
`python3 wf.py gabbagabbahey`

Exercise 19 Run the unedited `wf.py` on a small file

Before you edit `wf.py` (see below), run the script on a small test file, `texts/trsc.txt` and save the output to a file.

1. `python3 wf.py texts/trsc.txt > unedit.txt`

Exercise 20 First edit of wf.py (1)

There are some lines of code in the script that are currently disabled. These are lines preceded by a hash sign, #. Here is one of them, on line 18:

```
# 1:  
#t = t.lower()
```

1. Use `micro` to edit `wf.py`:
remove the hash sign on line 18, to enable some new code to run:

```
# 1:  
t = t.lower()
```

Exercise 21 Run the edited `wf.py`

1. Can you guess what the edit above will change?
2. Run the edited script and save the output:

```
python3 wf.py texts/trsc.txt > edit.txt
```
3. Compare `unedit.txt` (see above) to `edit.txt`
What differs?

Exercise 22 Second edit of wf.py (2)

Edit wf.py once again, but this time remove the # (comment characters) from lines 21 and 22:

```
#if t in stopwords:  
#    continue
```

1. Run the newest edited version and save the result:
`python3 wf.py texts/trsc.txt > second_edit.txt`
2. Compare second_edit.txt to edit.txt (see above). What's the difference?

Exercise 23 Edit the print statement

What happens if you edit the print statement at the end, and swap places of 1 and 0?

Change

```
print(f"{tup[1]}\t{tup[0]}")
```

into

```
print(f"{tup[0]}\t{tup[1]}")
```

and run wf.py again.

Exercise 24 The outer loop (1)

Copy the scripts

```
~kali2818/scripts/outer_loop_wf_1.py
```

and

```
~kali2818/scripts/outer_loop_wf_2.py
```

1. Compare the two scripts above, how do they differ?
2. Run both scripts with some arguments (e.g. `arg1 arg2`)
3. How does the output differ?
4. What if you run them without any arguments?

Exercise 25 The outer loop (2)

1. Change the file `outer_loop_wf_1.py` so that it down-cases any upper case arguments:

```
python3 outer_loop_wf_1.py HELLO  
hello
```

Hint: Be inspired by the first edit of `wf.py` above (`t = t.lower()`)

Exercise 26 Advanced: figure out word frequencies pipeline

Only consider this if you are done with the rest of the exercises. You don't need to spend time on this if you're not interested

Try to *understand* the following pipeline (see last page of `linux_commands.pdf`):

```
cat file.txt|egrep -o '\w+'|sort|uniq -c|sort -rn
```

Tip 1: Break it apart. Build up the pipeline piece by piece, one command at a time.

Tip 2: Use some simple input that you understand (rather than some large file). For example, create a file with the following contents:

```
d b c a b c a c a a
```

(Now you can more easily compare the output of the different parts of the pipeline to the original input.)

Tip 3: Check the `man` pages (or online documentation) for the switches to the commands